

# CPSC 250 / 250L

## Command Line Git Reference

A lecture-style summary with examples

### Big idea

Git is a version-control system. It keeps a history of your work, lets you move safely between versions, and connects your local copy of a project to a shared copy on GitHub. In this course, the usual workflow is:

edit files → stage files → commit locally → push to GitHub.

## Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>The mental model</b>                                                | <b>2</b> |
| <b>2</b> | <b>One-time setup commands</b>                                         | <b>2</b> |
| 2.1      | Configure your name and email . . . . .                                | 2        |
| 2.2      | Clone a repository . . . . .                                           | 2        |
| 2.3      | See the connected remotes . . . . .                                    | 3        |
| 2.4      | Add the instructor repository as upstream . . . . .                    | 3        |
| 2.5      | Change a remote URL . . . . .                                          | 3        |
| <b>3</b> | <b>The daily save-and-submit workflow</b>                              | <b>3</b> |
| 3.1      | Check the current state . . . . .                                      | 3        |
| 3.2      | Stage files . . . . .                                                  | 3        |
| 3.3      | Commit staged files . . . . .                                          | 4        |
| 3.4      | Push your commits to GitHub . . . . .                                  | 4        |
| <b>4</b> | <b>Getting updates from GitHub</b>                                     | <b>4</b> |
| 4.1      | Fetch without changing your files . . . . .                            | 4        |
| 4.2      | Pull from your own GitHub fork . . . . .                               | 4        |
| 4.3      | Update your local main branch from the instructor repository . . . . . | 4        |
| <b>5</b> | <b>Looking at history and changes</b>                                  | <b>5</b> |
| 5.1      | View the commit history . . . . .                                      | 5        |
| 5.2      | See unstaged changes . . . . .                                         | 5        |
| 5.3      | See staged changes . . . . .                                           | 5        |
| <b>6</b> | <b>Branches</b>                                                        | <b>5</b> |
| 6.1      | List branches . . . . .                                                | 5        |
| 6.2      | Create and switch to a new branch . . . . .                            | 6        |
| 6.3      | Switch between existing branches . . . . .                             | 6        |
| 6.4      | Merge a feature branch into main . . . . .                             | 6        |
| 6.5      | Delete a finished branch . . . . .                                     | 6        |

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| <b>7 Undoing and fixing mistakes</b>                                               | <b>6</b> |
| 7.1 Unstage a file . . . . .                                                       | 6        |
| 7.2 Discard local changes to a file . . . . .                                      | 7        |
| 7.3 Fix the most recent commit message . . . . .                                   | 7        |
| 7.4 Revert a committed change safely . . . . .                                     | 7        |
| <b>8 Merge conflicts</b>                                                           | <b>7</b> |
| 8.1 Typical conflict workflow . . . . .                                            | 7        |
| <b>9 Files Git should ignore</b>                                                   | <b>7</b> |
| <b>10 Common course scenarios</b>                                                  | <b>8</b> |
| 10.1 Scenario 1: Submit today's lab . . . . .                                      | 8        |
| 10.2 Scenario 2: Get the latest lab files from the instructor repository . . . . . | 8        |
| 10.3 Scenario 3: Start a new lab branch . . . . .                                  | 8        |
| 10.4 Scenario 4: Merge your lab branch back into main . . . . .                    | 8        |
| 10.5 Scenario 5: Diagnose a weird repository problem . . . . .                     | 9        |
| <b>11 Command summary table</b>                                                    | <b>9</b> |
| <b>12 A final checklist before asking for help</b>                                 | <b>9</b> |

# 1 The mental model

A Git project has several important places where files can live.

| Place             | Meaning                                                                                                                                                  |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Working directory | The actual files you are editing in PyCharm, VS Code, or a terminal.                                                                                     |
| Staging area      | A preparation area for the next commit. Files get here with <code>git add</code> .                                                                       |
| Local repository  | The saved history on your own computer. New snapshots get here with <code>git commit</code> .                                                            |
| Remote repository | The GitHub copy of the repository. Your fork is usually called <code>origin</code> . The instructor repository is usually called <code>upstream</code> . |

## Course vocabulary

- **Repository:** a folder tracked by Git.
- **Commit:** a saved snapshot of the project.
- **Branch:** a named line of development.
- **Fork:** your GitHub copy of someone else's repository.
- **Remote:** a nickname for a GitHub repository connected to your local repository.
- **Merge:** combining changes from one branch or repository into another.

## 2 One-time setup commands

These commands are usually done once on a computer or once for a repository.

### 2.1 Configure your name and email

Git records the author of each commit. The name and email should identify you.

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada.lovelace.24@cnu.edu"
```

Check your settings:

```
git config --global --list
```

### 2.2 Clone a repository

`git clone` downloads a repository from GitHub and creates a local working copy.

```
git clone https://github.com/YOUR-USERNAME/cpsc250L.git
```

Then move into the folder:

```
cd cpsc250L
```

## 2.3 See the connected remotes

`git remote -v` shows the GitHub repositories connected to your local repository.

```
git remote -v
```

Typical course output:

```
origin    https://github.com/YOUR-USERNAME/cpsc250L.git (fetch)
origin    https://github.com/YOUR-USERNAME/cpsc250L.git (push)
upstream  https://github.com/brash99/cpsc250L.git (fetch)
upstream  https://github.com/brash99/cpsc250L.git (push)
```

## 2.4 Add the instructor repository as upstream

If your local repository only knows about your fork, add the instructor repository as **upstream**.

```
git remote add upstream https://github.com/brash99/cpsc250L.git
```

## 2.5 Change a remote URL

Sometimes a remote points to the wrong GitHub repository. Change it with `git remote set-url`.

```
git remote set-url upstream https://github.com/brash99/cpsc250L-instructor.git
```

Then verify:

```
git remote -v
```

# 3 The daily save-and-submit workflow

## 3.1 Check the current state

The most important Git command is `git status`. Use it constantly.

```
git status
```

Common messages:

| Message                               | Meaning                                                           |
|---------------------------------------|-------------------------------------------------------------------|
| nothing to commit, working tree clean | Git sees no unsaved changes.                                      |
| Changes not staged for commit         | Files changed, but they are not yet prepared for the next commit. |
| Untracked files                       | Git sees new files that have not yet been added.                  |
| Changes to be committed               | Files are staged and ready to commit.                             |

## 3.2 Stage files

To stage one file:

```
git add labs/lab03_functions/solution.py
```

To stage every changed file in the current repository:

```
git add .
```

### Important habit

Before using `git add .`, run `git status`. Make sure you are not accidentally adding temporary files, large data files, or virtual-environment folders.

## 3.3 Commit staged files

A commit saves a snapshot in your local repository.

```
git commit -m "Complete lab 03 functions"
```

Good commit messages are short, specific, and written in the imperative style:

```
git commit -m "Add CSV parsing for lab 04"
git commit -m "Fix Plant __str__ method"
git commit -m "Refactor book search into helper function"
```

## 3.4 Push your commits to GitHub

A commit is still only on your computer until you push it.

```
git push
```

The first push from a new branch may need:

```
git push -u origin lab05-feature
```

After that, `git push` is usually enough.

# 4 Getting updates from GitHub

## 4.1 Fetch without changing your files

`git fetch` contacts GitHub and downloads information about new commits, but it does not merge them into your current branch.

```
git fetch upstream
```

This is a safe way to ask, “What has changed in the instructor repository?”

## 4.2 Pull from your own GitHub fork

`git pull` fetches and then merges changes into your current branch.

```
git pull
```

This is commonly used when your GitHub fork has changes that your local computer does not yet have.

## 4.3 Update your local main branch from the instructor repository

A common course pattern is:

```
git switch main
git fetch upstream
git merge upstream/main
```

This says:

1. Move to your local `main` branch.
2. Get the latest information from the instructor repository.
3. Merge the instructor's `main` branch into your local `main` branch.

Then push the updated version of your fork:

```
git push origin main
```

## 5 Looking at history and changes

### 5.1 View the commit history

```
git log
```

A compact version is often easier to read:

```
git log --oneline
```

A useful branch graph view:

```
git log --oneline --graph --decorate --all
```

### 5.2 See unstaged changes

`git diff` shows changes in the working directory that have not been staged.

```
git diff
```

### 5.3 See staged changes

After `git add`, use:

```
git diff --staged
```

This answers: “What exactly am I about to commit?”

## 6 Branches

Branches allow us to work on a feature or lab without immediately changing `main`.

### 6.1 List branches

```
git branch
```

Show local and remote branches:

```
git branch -a
```

## 6.2 Create and switch to a new branch

```
git switch -c lab05-feature
```

Older Git examples often use `checkout` instead:

```
git checkout -b lab05-feature
```

## 6.3 Switch between existing branches

```
git switch main  
git switch lab05-feature
```

Older style:

```
git checkout main
```

## 6.4 Merge a feature branch into main

```
git switch main  
git merge lab05-feature
```

Then push:

```
git push origin main
```

## 6.5 Delete a finished branch

After the branch has been merged:

```
git branch -d lab05-feature
```

Force-delete a branch only when you are sure you do not need it:

```
git branch -D lab05-feature
```

# 7 Undoing and fixing mistakes

## 7.1 Unstage a file

If you staged a file with `git add` but do not want it in the next commit:

```
git restore --staged labs/lab04_csv/solution.py
```

Older Git examples may use:

```
git reset HEAD labs/lab04_csv/solution.py
```

## 7.2 Discard local changes to a file

This throws away uncommitted changes to a file and restores the last committed version.

```
git restore labs/lab04_csv/solution.py
```

### This is destructive

Only use `git restore FILE` when you really want to lose the uncommitted edits in that file.

## 7.3 Fix the most recent commit message

```
git commit --amend -m "Complete lab 04 CSV processing"
```

Use this before pushing when possible. Amending a commit that has already been pushed can confuse shared history.

## 7.4 Revert a committed change safely

`git revert` creates a new commit that undoes an earlier commit.

```
git log --oneline
git revert abc1234
```

This is safer than rewriting history because it preserves the record of what happened.

# 8 Merge conflicts

A merge conflict happens when Git cannot automatically combine two versions of a file.

## 8.1 Typical conflict workflow

```
git status
# Open the conflicted file and edit it by hand.
git add conflicted_file.py
git commit
```

Conflict markers look like this:

```
<<<<<< HEAD
print("My version")
=====
print("Other version")
>>>>>> upstream/main
```

Your job is to edit the file into the version that should remain, then remove the conflict markers.

# 9 Files Git should ignore

Some files do not belong in the repository: virtual environments, cache files, temporary files, and generated output. Put patterns for these files in `.gitignore`.

Example `.gitignore` entries for Python work:



```
.venv/  
venv/  
__pycache__/  
*.pyc  
.DS_Store  
.idea/
```

After editing `.gitignore`, commit it like any other file:

```
git add .gitignore  
git commit -m "Update Python gitignore rules"  
git push
```

## 10 Common course scenarios

### 10.1 Scenario 1: Submit today's lab

```
git status  
git add labs/lab04_csv_processing/solution.py  
git status  
git commit -m "Complete lab 04 CSV processing"  
git push
```

### 10.2 Scenario 2: Get the latest lab files from the instructor repository

```
git switch main  
git status  
git fetch upstream  
git merge upstream/main  
git push origin main
```

### 10.3 Scenario 3: Start a new lab branch

```
git switch main  
git pull  
git switch -c lab05-classes
```

Work on the lab, then save it:

```
git add labs/lab05_classes_feature_branches/  
git commit -m "Complete lab 05 classes"  
git push -u origin lab05-classes
```

### 10.4 Scenario 4: Merge your lab branch back into main

```
git switch main  
git merge lab05-classes  
git push origin main
```

## 10.5 Scenario 5: Diagnose a weird repository problem

Start with read-only commands:

```
git status
git branch -a
git remote -v
git log --oneline --graph --decorate --all
```

These commands do not change the repository. They help you see where you are before deciding what to do.

## 11 Command summary table

| Command                                            | Purpose                                                              |
|----------------------------------------------------|----------------------------------------------------------------------|
| <code>git config -global user.name "Name"</code>   | Set the author name for commits.                                     |
| <code>git config -global user.email "email"</code> | Set the author email for commits.                                    |
| <code>git clone URL</code>                         | Download a repository from GitHub.                                   |
| <code>git status</code>                            | Show changed, staged, and untracked files.                           |
| <code>git add FILE</code>                          | Stage a file for the next commit.                                    |
| <code>git add .</code>                             | Stage all changes in the current repository.                         |
| <code>git commit -m "message"</code>               | Save a local snapshot.                                               |
| <code>git push</code>                              | Upload local commits to GitHub.                                      |
| <code>git pull</code>                              | Download and merge changes from the default remote branch.           |
| <code>git fetch upstream</code>                    | Download information from the instructor repository without merging. |
| <code>git merge upstream/main</code>               | Merge instructor changes into the current branch.                    |
| <code>git remote -v</code>                         | List connected GitHub repositories.                                  |
| <code>git remote add upstream URL</code>           | Add the instructor repository as a remote.                           |
| <code>git remote set-url upstream URL</code>       | Change where a remote points.                                        |
| <code>git branch</code>                            | List local branches.                                                 |
| <code>git branch -a</code>                         | List local and remote branches.                                      |
| <code>git switch BRANCH</code>                     | Move to an existing branch.                                          |
| <code>git switch -c BRANCH</code>                  | Create and move to a new branch.                                     |
| <code>git checkout BRANCH</code>                   | Older command for switching branches.                                |
| <code>git checkout -b BRANCH</code>                | Older command for creating and switching to a branch.                |
| <code>git log -oneline</code>                      | Show compact commit history.                                         |
| <code>git diff</code>                              | Show unstaged changes.                                               |
| <code>git diff -staged</code>                      | Show staged changes.                                                 |
| <code>git restore -staged FILE</code>              | Unstage a file.                                                      |
| <code>git restore FILE</code>                      | Discard uncommitted changes to a file.                               |
| <code>git commit -amend</code>                     | Modify the most recent commit.                                       |
| <code>git revert HASH</code>                       | Safely undo an old commit by creating a new commit.                  |
| <code>git branch -d BRANCH</code>                  | Delete a merged local branch.                                        |

## 12 A final checklist before asking for help

When Git seems confusing, run these commands and read the output carefully:

```
git status
git branch -a
git remote -v
```

```
git log --oneline --graph --decorate --all
```

Then ask:

1. What branch am I on?
2. Are my files changed, staged, committed, or pushed?
3. Is `origin` my fork?
4. Is `upstream` the instructor repository?
5. Am I trying to save my work, submit my work, or get new instructor files?

### The safest Git habit

Use `git status` before and after every important Git command. Git usually tells you exactly what state the repository is in and what command might make sense next.